

AR-Desk

NQ/MNQ 日内盘前决策辅助系统。每个交易日美东 09:20 自动运行一次，产出一份带 **日型分类 / 入场区 / 目标位阶梯 / 不做条件 / 仓位 / 收工规则** 的盘前简报，推送到 Telegram。

这套东西的定位

它不下单，也不告诉你"现在买"。它做三件事：

1. **把开盘前能算的东西全部算完** —— 亚洲/伦敦区间、日型、目标位阶梯、折价区、失效线、手数。
2. **把"不做"的条件变成硬性检查** —— 每一项不通过就是不通过，不给你临场找理由的空间。
3. **把计划落盘** —— 每天的计划自动写进 `data/journal.csv`，收盘后你回填结果，跑够 30-40 笔之后 `--stats` 会告诉你哪种日型真的赚钱、哪个 gate 其实在帮倒忙。

盘中那两步脚本不替你做：5 分钟参与度、footprint 上的吸筹 + 主导权翻转 + 价格推进。简报会把门槛写出来，确认还是你自己在 ATAS 上做。

快速开始

```
pip install -r requirements.txt
python main.py --force           # 立刻跑一次
python main.py --date 2026-09-02 # 复盘某一天
python main.py --stats           # 按日型看历史表现
```

GitHub Actions：把 repo push 上去，在 Settings → Secrets 加 `TELEGRAM_BOT_TOKEN` 和 `TELEGRAM_CHAT_ID`，工作流已经挂好 13:20 和 14:20 UTC 两个 cron（夏令时/冬令时各一个），脚本内部用美东时间自己判断该不该跑。

结构

<code>main.py</code>	编排：拉数据 → 分段 → 分类 → 目标 → 检查 → 出简报
<code>config.yaml</code>	所有阈值，改这里就够了
<code>src/data.py</code>	K 线、时段切片、ATR、分形摆动点
<code>src/engine.py</code>	日型分类、目标位阶梯、折价/溢价区、不做条件
<code>src/gex.py</code>	naive GEX（波动环境，不是方向信号）
<code>src/risk.py</code>	风控状态机 + 手数计算

src/brief.py 简报渲染
src/journal.py 计划落盘、日型统计、Telegram

三个需要你self接的地方

1. GEX 数据源。 默认 `gex.source: file`，读 `data/gex_latest.json` ——直接把你 `mikeli008008/gex` 那套 CBOE 输出写到这个路径就行。读不到会回退到 `yfinance` 的 `QQQ` 期权链算 naive GEX，但那个数据在盘后经常残缺，代码会自己降级成"环境未知、按中性处理"，不会给你一个错的墙位。

2. 行情源。 现在用 `yfinance` 的 `NQ=F`（5m 最多回溯 60 天，够用但不是干净的连续合约）。要更准就把 `src/data.py` 的 `get_bars` 换成 `Databento / Tradovate` 拉真实 MNQ 连续合约，其余代码不用动。

3. 阈值标定。 `classification.reclaim_threshold` 默认 0.5（伦敦扫边后要收回半个亚洲区间才算反转燃料）。这个值偏严——最近 7 个交易日里 5 天被判成轮转日。先用 `--date` 把过去两三个月一天天跑一遍，看哪个值能把你实际赚钱的那些天分出来，再定死。**不要先上仓位再标定。**

已知边界

- 日型分类是对形态的机械翻译，不是对"今天会不会走出来"的预测。轮转日多是正常的。
- 目标位阶梯只排序不预测：最近的那个先成为目标，不代表一定会到。
- 手数按固定百分比风险算，没有考虑 `prop` 账户的 `trailing drawdown` 曲线——`Lucid` 那种回撤跟随的规则，你得在 `risk.size_position` 里加一层自己的约束。
- 统计模块在样本 < 30 笔时的数字没有意义，别看。

本项目只做上下文整理和纪律约束，不构成投资建议。